



*Campus
Mexicali*

**“Las ciencias
Básicas son el
cimiento de la
ingeniería”**

20-Mayo-2026

INSTITUTO TECNOLÓGICO DE MEXICALI

Carrera:

ING. SISTEMAS COMPUTACIONALES.

Materia:

POO

Tema:

ARCHIVOS U6.

Alumnos:

Monge Delmoral José Manuel

Profesor(a):

Marisela Ponce Millanes.

La Clase `File` en Java: Manejo de Archivos y Directorios

En Java, la clase `File` (perteneciente al paquete `java.io`) es una representación abstracta de las rutas de archivos y directorios del sistema de archivos de tu computadora. No se utiliza para leer o escribir el contenido de un archivo directamente, sino para gestionar sus **propiedades, metadatos y existencia**.

Tipos de Elementos que Maneja

La clase `File` no distingue inicialmente si una ruta es un archivo con texto o una carpeta; trata a ambos como objetos interconectados con el sistema operativo. Se dividen principalmente en:

- **Archivo (File):** Apunta a un elemento contenedor de datos específico (como un `.txt`, `.jpg` o `.pdf`). Java ofrece métodos booleanos para comprobar que la ruta pertenece efectivamente a un archivo físico de datos y no a una carpeta.
- **Directorio (Directory):** Es una ruta contenedora o carpeta. No almacena datos directamente en sí misma, sino que sirve para organizar y guardar múltiples archivos u otros subdirectorios.

Usos y Operaciones Principales

Esta clase es la una muy versátil en java para interactuar con el disco duro. Permite realizar las siguientes operaciones esenciales:

1. Creación y Modificación

- **Crear un archivo desde cero:** Si la ruta especificada no existe, permite generar un nuevo archivo en el disco.
- **Crear directorios:** Puede crear una carpeta simple o incluso una estructura completa de carpetas anidadas (rutas con múltiples subcarpetas).

2. Comprobación y Estado

- **Verificar existencia:** Permite validar si un archivo o carpeta realmente existe en el disco antes de intentar abrirlo, evitando errores en el programa.
- **Diferenciación de tipo:** Comprueba mediante métodos lógicos si el objeto en cuestión es un archivo ejecutable/de datos o un directorio.

3. Seguridad y Permisos

- **Verificar permisos de lectura:** Consulta si el programa tiene autorización para abrir y leer el archivo.
- **Verificar permisos de escritura:** Consulta si el sistema operativo permite modificar o añadir información al elemento.

4. Destrucción y Limpieza

- **Borrar ficheros o carpetas:** Permite eliminar archivos del sistema de forma definitiva (o carpetas, siempre y cuando estén vacías).

Archivos Binarios Puros (Creación, Escritura y Lectura)

Para interactuar con archivos binarios a bajo nivel (byte por byte), Java utiliza el paquete `java.io` mediante las clases `FileOutputStream` y `FileInputStream`. Para escribir tipos de datos primitivos (`int`, `double`, `boolean`) de forma directa, estas clases se envuelven en `DataOutputStream` y `DataInputStream`.

1. Cómo se crea y escribe en un archivo binario

El archivo se crea automáticamente en el disco en el momento en que se inicializa el flujo de salida (`FileOutputStream`). Si el archivo ya existe, por defecto se sobrescribe su contenido.

Java

```
import java.io.DataOutputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class EscrituraBinaria {
    public static void main(String[] args) {
        // El bloque try-with-resources crea el archivo y asegura el
        // cierre del flujo
        try (DataOutputStream dos = new DataOutputStream(new
FileOutputStream("datos.dat"))) {
            // Escritura de datos primitivos en formato binario
            dos.writeInt(1024);
            dos.writeDouble(95.5);
            dos.writeBoolean(true);
        } catch (IOException e) {
            System.out.println("Error en la creación/escritura del
archivo: " + e.getMessage());
        }
    }
}
```

```
}
```

2. Cómo se lee un archivo binario

Para recuperar los datos, el flujo de entrada debe leer los elementos **en el mismo orden exacto** en el que fueron escritos; de lo contrario, las posiciones de los bytes se desalinean y los datos se corrompen.

Java

```
import java.io.DataInputStream;
import java.io.FileInputStream;
import java.io.IOException;

public class LecturaBinaria {
    public static void main(String[] args) {
        try (DataInputStream dis = new DataInputStream(new
FileInputStream("datos.dat"))) {
            // Lectura secuencial estricta
            int numero = dis.readInt();
            double calificacion = dis.readDouble();
            boolean activo = dis.readBoolean();

            System.out.println("Datos recuperados: " + numero + ", " +
calificacion + ", " + activo);
        } catch (IOException e) {
            System.out.println("Error en la lectura del archivo: " +
e.getMessage());
        }
    }
}
```

Archivos de Objetos (Serialización)

Un **archivo de objetos** es un tipo especial de archivo binario que almacena el estado completo de una instancia de una clase (sus variables y relaciones) directamente en el disco.

Para que un objeto pueda ser convertido en un flujo de bytes y guardado, su clase constructora debe implementar la interfaz **java.io.Serializable**. Esta interfaz es un marcador lógico: no tiene métodos, solo le otorga al compilador el "permiso" para desarmar el objeto en bytes.

1. Estructura de la Clase a Guardar

Java

```
import java.io.Serializable;

// Regla de negocio: Debe ser Serializable

public class Alumno implements Serializable {
    // ID de control de versión para asegurar compatibilidad del archivo
    private static final long serialVersionUID = 1L;

    private String nombre;
    private int edad;

    public Alumno(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    @Override
    public String toString() {
        return "Alumno: " + nombre + " | Edad: " + edad;
    }
}
```

2. Cómo se escribe un Objeto en el disco (Serialización)

Se utiliza la clase wrapper `ObjectOutputStream` y su método `writeObject()`.

Java

```
import java.io.FileOutputStream;
import java.io.ObjectOutputStream;
import java.io.IOException;

public class GuardarObjeto {
    public static void main(String[] args) {
        Alumno alumno = new Alumno("Jose Manuel", 19);

        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("alumno.ser"))) {
            oos.writeObject(alumno); // Guarda el objeto completo de un
solo golpe
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

3. Cómo se lee un Objeto desde el disco (Deserialización)

Se utiliza `ObjectInputStream` y su método `readObject()`. El método devuelve un tipo genérico `Object`, por lo que es obligatorio aplicar un *cast* explícito al tipo de clase original.

Java

```
import java.io.FileInputStream;
import java.io.ObjectInputStream;
import java.io.IOException;

public class LeerObjeto {
    public static void main(String[] args) {
```

```
        try (ObjectInputStream ois = new ObjectInputStream(new  
FileInputStream("alumno.ser"))) {
```

```
            Alumno alumnoRecuperado = (Alumno) ois.readObject(); //  
Lectura y Cast
```

```
            System.out.println(alumnoRecuperado);
```

```
        } catch (IOException | ClassNotFoundException e) {
```

```
            System.out.println("Error al reconstruir el objeto: " +  
e.getMessage());
```

```
        }
```

```
    }
```

```
}
```

Análisis Comparativo y Buenas Prácticas

1. Diferencia entre Archivo Binario Puro y Archivo de Objetos

- **Archivo Binario Puro (`DataStream`):** Almacena únicamente los valores crudos de los datos (`1024, 95.5`). Requiere que el programador conozca la estructura exacta del archivo para reconstruirlo manualmente al leer. Es altamente eficiente y compatible con otros lenguajes de programación.
- **Archivo de Objetos (`ObjectStream`):** Además de los datos crudos de las variables, almacena los metadatos del objeto (el nombre de la clase, el paquete, la firma de la estructura). Java se encarga de rearmar la instancia automáticamente. Es exclusivo del ecosistema Java y genera archivos ligeramente más pesados.

2. Ventajas de manejar Archivos en Java

- **Persistencia Real:** Permite que los datos de tus sistemas (como los registros de tu veterinaria o la configuración de tu bot) sobrevivan cuando el programa se apaga o la computadora se reinicia.
- **Liberación de Memoria RAM:** Los datos masivos se pueden almacenar secuencialmente en el disco duro, evitando saturar la memoria de procesamiento activa de la aplicación.

3. Desventajas del uso de Archivos

- **Velocidad de Entrada/Salida (I/O Bottleneck):** Leer y escribir en el disco físico es miles de veces más lento que procesar los datos directamente en la memoria RAM.
- **Acceso Secuencial de Bajo Nivel:** Encontrar un dato específico en un archivo binario grande requiere leer todo el archivo desde el principio hasta llegar al punto deseado, a diferencia de una base de datos relacional que cuenta con indexación eficiente.

4. Errores Frecuentes al manejar Archivos (Bloques de Excepción)

- **`FileNotFoundException`:** Intentar leer un archivo con `FileInputStream` usando una ruta que no existe en el almacenamiento real. Se previene corriendo primero el método `file.exists()` de la clase `File`.
- **`EOFException` (End of File):** Ocurre cuando el programa intenta seguir ejecutando métodos de lectura (`readInt()`, `readObject()`) cuando el flujo ya ha llegado al final físico del archivo.
- **`NotSerializableException`:** El error clásico al trabajar con objetos. Ocurre cuando intentas usar `writeObject()` con una clase que no tiene el implemento explícito de la interfaz `Serializable`.

- **Olvidar cerrar los Flujos (Fuga de Recursos):** Si un archivo no se cierra mediante el método `.close()` o mediante la sintaxis de un bloque *try-with-resources*, el archivo queda bloqueado a nivel de sistema operativo, impidiendo que otros procesos lo editen o lo borren hasta que la máquina virtual de Java se apague por completo.